

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0508

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr.G.Ayyappan

Schema Used in All Questions:

Student (RollNo, Name, DeptID, Age, Marks) | Department (DeptID, DeptName, HOD)

Sample Data (load this into MySQL before attempting the questions):

Department

DeptID	DeptName	HOD
1	Computer Science	Dr. Rao
2	Electronics	Dr. Iyer
3	Mechanical	Dr. Nair

Student

RollNo	Name	DeptID	Age	Marks
101	Aravind	1	20	85
102	Bhavya	1	21	78
103	Chandran	1	20	90

104	Deepak	2	22	60
105	Esha	2	21	55
106	Farhan	2	22	95
107	Gowri	3	20	40
108	Hari	3	21	45
109	Indra	3	20	68

QUESTIONS Total Marks: 50			
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results. Given Query: <pre>SELECT s.Name, d.DeptName FROM Student s, Department d; -- Intent: list each student with their own department's name</pre>	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly. Given Query: <pre>SELECT s.Name, s.DeptID, s.Marks FROM Student s WHERE s.Marks > (SELECT AVG(Marks) FROM Student); -- Intent: list students who score above their OWN department's average</pre>	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view. Given Query: <pre>CREATE VIEW DeptAvgMarks AS SELECT Name, DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY DeptID; SELECT * FROM DeptAvgMarks;</pre>	BL3 – Apply	5

4	Debug the given relational algebra expression that produces incorrect output for a natural join operation. Given Query: $\pi(\text{Name, Age}) (\text{Student}) \bowtie \text{Department}$ -- Intent: natural join of Student and Department on DeptID	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints. Given Query: <pre>CREATE TABLE Student (RollNo INT PRIMARY KEY, Name VARCHAR(50), DeptID INT, Age INT, Marks INT);</pre> -- Department table only has DeptID values 1, 2, 3 <pre>INSERT INTO Student (RollNo, Name, DeptID, Age, Marks) VALUES (110, 'Arun', 9, 20, 78);</pre>	BL3 – Apply	5

6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance. Given Query: <pre>SELECT s.Name, (SELECT d.DeptName FROM Department d WHERE d.DeptID = s.DeptID) AS DeptName FROM Student s;</pre>	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error. Given Query: <pre>SELECT DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY RollNo;</pre>	BL3 – Apply	5
8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table. Given Query: -- Intent: give 5 bonus marks to students in Department 2 only <pre>UPDATE Student SET Marks = Marks + 5;</pre>	BL3 – Apply	5

9	Optimize the given multi-table SQL query by restructuring sub queries into JOINS and using appropriate indexes. Given Query: <pre>SELECT s.Name, s.Marks, (SELECT DeptName FROM Department WHERE DeptID = s.DeptID) AS DeptName, (SELECT HOD FROM Department WHERE DeptID = s.DeptID) AS HeadOfDept FROM Student s WHERE s.DeptID IN (SELECT DeptID FROM Department WHERE DeptName IN ('Computer Science', 'Electronics'));</pre>	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage. Given Query: <pre>-- Intent: students who scored more than AT LEAST ONE student in Department 2 { s.Name Student(s) $\wedge$ ($\forall t$) (Student(t) $\wedge$ t.DeptID = 2 $\rightarrow$ s.Marks > t.Marks) }</pre>	BL3 – Apply	5

Code of Conduct:

*I **KESHAVARAM L/192511253**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

L. Keshavaram

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly

Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Solutions:

Q1: Fix incorrect JOIN syntax returning duplicate results

Problem Identification (20%): The provided SQL query ``SELECT s.Name, d.DeptName FROM Student s, Department d;`` attempts an old-style implicit join without a WHERE clause to restrict the relationship. This results in a Cartesian product (Cross Join), where every row in the Student table is joined with every row in the Department table, returning $N \times M$ rows (duplicate/incorrect assignments).

Debugging (25%): The debugging process involves replacing the comma-separated FROM clause with an explicit ANSI-standard INNER JOIN. This explicitly requires an ON clause. The linking foreign-key relationship between the two tables is DeptID.

Optimization (20%): By switching to an explicit JOIN with an ON condition, the query

optimizer can utilize indexes on 'DeptID' if they exist. This reduces the time complexity from $O(N * M)$ (cross product space) to roughly $O(N \log M)$ or better depending on indexing (e.g., hash join or merge join).

Analysis & Reasoning (20%): Logically, a student belongs to exactly one department. The implicit comma syntax is prone to accidental cross joins if the WHERE clause is forgotten. Explicit JOIN syntax strictly separates join logic from filter logic, improving maintainability and avoiding this entire class of logical errors.

Code Quality & Solution (15%):

-- Optimized and Corrected Query

```
SELECT
s.Name,
d.DeptName
FROM
Student s
INNER JOIN
Department d ON s.DeptID = d.DeptID;
```

Q2: Rewrite sub-query missing a correlation

Problem Identification (20%): The subquery '(SELECT AVG(Marks) FROM Student)' operates independently of the outer query. It computes the global average of ALL students across ALL departments. The intent, however, is to compare a student's marks against their *own* department's average, which requires a Correlated Subquery.

Debugging (25%): To resolve this, an alias is added to the table in the subquery ('Student s2'). A correlation condition 'WHERE s2.DeptID = s.DeptID' is then introduced. This binds the inner query to the current row being evaluated by the outer query.

Optimization (20%): While a correlated subquery executes once per row (N times), it correctly encapsulates the logic. To further optimize this for larger datasets, one could use Window Functions (e.g., 'AVG(Marks) OVER(PARTITION BY DeptID)'), which calculates the averages in a single pass without redundant nested loop execution.

Analysis & Reasoning (20%): The logic dictates that peer comparison must be grouped by department. Without the correlation, the condition evaluates students from low-scoring departments against high-scoring departments unfairly. Binding 'DeptID' restores the relational context intended by the requirements.

Code Quality & Solution (15%):

```
-- Optimized using Correlated Subquery (Matches specific curriculum requirements)
SELECT
    s.Name,
    s.DeptID,
    s.Marks
FROM
    Student s
WHERE
    s.Marks > (
        SELECT AVG(Marks)
        FROM Student s2
        WHERE s2.DeptID = s.DeptID
    );
```

Q3: Rewrite a correct and optimized view

Problem Identification (20%): The query `SELECT Name, DeptID, AVG(Marks)... GROUP BY DeptID;` violates SQL Standard aggregation rules. When using `GROUP BY`, every column in the `SELECT` list must either be part of the `GROUP BY` clause or wrapped in an aggregate function. 'Name' is neither, causing a semantic error because there are multiple names per DeptID.

Debugging (25%): The debugging step requires identifying what data the View is meant to represent. The view is named `DeptAvgMarks`, indicating it should represent department-level metrics, not student-level data. Therefore, the 'Name' column must be completely removed from the SELECT clause.

Optimization (20%): By correctly formatting the GROUP BY clause, the database can execute a hash aggregation or stream aggregation efficiently. Creating this view optimizes future developer workflows, as they can query pre-aggregated data without rewriting the complex GROUP BY statement.

Analysis & Reasoning (20%): A View acts as a virtual table. Including granular data (Name) alongside aggregated data (AVG) creates a logical contradiction regarding the grain of the result set. Stripping granular identifiers ensures the view accurately represents a 1-to-1 relationship with departments.

Code Quality & Solution (15%):

```
-- Corrected View Definition
CREATE VIEW DeptAvgMarks AS
SELECT
    DeptID,
```

```

AVG(Marks) AS AvgMarks
FROM
Student
GROUP BY
DeptID;
-- Test the View
SELECT * FROM DeptAvgMarks;

```

Q4: Debug relational algebra expression

Problem Identification (20%): The relational algebra expression $\pi(\text{Name, Age}) (\text{Student}) \bowtie \text{Department}$ applies the projection operation (π) on the Student relation BEFORE the natural join ($\bowtie$). The projection drops the 'DeptID' column. Consequently, the natural join has no common attributes to link on, resulting in a Cartesian product instead of a proper join.

Debugging (25%): According to the order of operations in relational algebra, the natural join relies on shared attributes (in this case, DeptID). The debugging step is to parenthesize the join operation so it executes first: $(\text{Student} \bowtie \text{Department})$. Only after the relations are correctly linked should the projection $\pi(\text{Name, Age})$ extract the requested columns.

Optimization (20%): Optimizing relational algebraic expressions often involves pushing projections down the tree. However, you can only push projections down if you retain the join keys. A highly optimized logical plan would be: $\pi(\text{Name, Age}) (\pi(\text{Name, Age, DeptID})(\text{Student}) \bowtie \pi(\text{DeptID})(\text{Department}))$, which minimizes intermediate memory usage.

Analysis & Reasoning (20%): Logically, natural joins are implicitly defined by intersecting schemas. If we remove the intersection (DeptID) prematurely, we destroy the relational integrity of the operation. This highlights the importance of preserving referential keys during intermediate algebraic steps.

Code Quality & Solution (15%):

Optimized Relational Algebra Expression:

$\pi\{\text{Name, Age}\} (\text{Student} \bowtie \text{Department})$

Q5: Enforce proper constraints for referential integrity Problem

Identification (20%): The schema provided lacks a FOREIGN KEY constraint on the

`Student` table. As a result, the `INSERT` statement successfully adds a student with `DeptID = 9`, even though Department 9 does not exist. This is a severe violation of Referential Integrity, creating 'orphan' records.

Debugging (25%): To fix this, the `CREATE TABLE Student` definition must be altered to include a `FOREIGN KEY` constraint that points `DeptID` in the Student table to the Primary Key `DeptID` in the Department table. This forces the database engine to validate all inserts and updates.

Optimization (20%): Enforcing constraints at the database schema level is heavily optimized by modern RDBMS engines. Checking foreign keys utilizes the primary key index on the parent table (`Department`), making validation extremely fast ($O(1)$ or $O(\log N)$). It prevents the need for slow, manual application-level data integrity checks.

Analysis & Reasoning (20%): Referential integrity ensures that relationships between tables remain consistent. If a student belongs to a department, that department must exist. Adding the constraint guarantees that the state of the database mathematically conforms to the defined relational model.

Code Quality & Solution (15%):

-- Corrected Schema with Referential Integrity

```
CREATE TABLE Student (  
  RollNo INT PRIMARY KEY,  
  Name VARCHAR(50),  
  DeptID INT,  
  Age INT,  
  Marks INT,  
  CONSTRAINT fk_department  
  FOREIGN KEY (DeptID)  
  REFERENCES Department(DeptID)  
);
```

-- The invalid INSERT will now be safely rejected by the DB.

Q6: Optimize inefficient nested loop query using JOIN Problem

Identification (20%): The given query uses a correlated subquery in the SELECT clause: `(SELECT d.DeptName FROM Department d WHERE d.DeptID = s.DeptID)`. This forces the database to execute a separate SELECT query for every single row in the Student table. This is known as the N+1 query problem and causes severe performance degradation.

Debugging (25%): The most systematic debugging step is to unnest the subquery and convert it into a standard table JOIN. By moving the Department table into the FROM clause and linking it via `ON s.DeptID = d.DeptID`, we flatten the query structure. **Optimization (20%):** Replacing the correlated subquery with an INNER JOIN allows the relational query optimizer to load the tables efficiently. Instead of executing N nested loops, the optimizer can perform a single Hash Join or Merge Join, improving the time complexity significantly and drastically reducing I/O overhead.

Analysis & Reasoning (20%): Subqueries in the SELECT clause should generally be avoided for linking structural tables. JOINS are the mathematically appropriate tool for relational combinations. This ensures scalable architecture, where query execution time grows logarithmically rather than exponentially as the student population increases.

Code Quality & Solution (15%):

-- Optimized JOIN Query

```
SELECT
  s.Name,
  d.DeptName
FROM
  Student s
  INNER JOIN
  Department d ON d.DeptID = s.DeptID;
```

Q7: Debug GROUP BY query producing wrong aggregation

Problem Identification (20%): The query attempts to find the average marks but groups by `RollNo`. Since `RollNo` is the primary key and is completely unique for every student, every 'group' consists of exactly one row. Thus, `AVG(Marks)` just returns the individual student's marks, failing the intent of the aggregate function.

Debugging (25%): The error lies in the granularity of the grouping. To fix the logical error, we must identify the intended category. The `SELECT` clause retrieves `DeptID`, indicating we want the average per department. The debugging fix is to replace `GROUP BY RollNo` with `GROUP BY DeptID`.

Optimization (20%): Grouping by a high-cardinality column (like a Primary Key) forces the database to allocate memory for N groups, which is inefficient and wasteful for aggregations. Changing to the low-cardinality `DeptID` column allows the aggregation engine to operate in

an optimized, constrained memory footprint.

Analysis & Reasoning (20%): Aggregate functions summarize data. Grouping by a unique identifier defeats the purpose of summarization. Proper relational grouping requires identifying the categorical dimension (Department) and applying the mathematical reduction (Average) across the continuous measure (Marks) within that specific dimension.

Code Quality & Solution (15%):

-- Corrected Aggregation Query

```
SELECT
  DeptID,
  AVG(Marks) AS AvgMarks
FROM
  Student
GROUP BY
  DeptID;
```

Q8: Fix UPDATE statement to avoid unintentional modification

Problem Identification (20%): The provided `UPDATE Student SET Marks = Marks + 5;` lacks a `WHERE` clause. In SQL, an UPDATE statement without a WHERE clause applies the mutation to every single row in the entire table. This inadvertently gives 5 bonus marks to all departments, not just Department 2.

Debugging (25%): To resolve this destructive error, a filtering condition must be appended immediately. Adding `WHERE DeptID = 2;` restricts the scope of the operation. In production environments, running updates without WHERE clauses can cause catastrophic data corruption.

Optimization (20%): Using a WHERE clause not only fixes the logic but is a massive optimization. It allows the database engine to use an index on `DeptID` to instantly locate only the affected rows, applying row-level locks only where necessary, rather than executing a full table scan and locking the entire table.

Analysis & Reasoning (20%): Data manipulation (DML) requires precise targeting. Logical errors in DML statements are dangerous because they permanently alter state. The principle of least privilege in data manipulation demands that updates are constrained to the tightest possible relational boundary.

Code Quality & Solution (15%):

-- Corrected and Safe UPDATE Statement

```
UPDATE Student
```

SET Marks = Marks + 5

WHERE DeptID = 2;

Q9: Optimize multi-table query

Problem Identification (20%): The query suffers from extreme nesting: two separate correlated subqueries in the `SELECT` clause to fetch `DeptName` and `HOD`, plus an `IN` subquery in the `WHERE` clause. This results in the database scanning the `Department` table three distinct times per student row. This is highly redundant.

Debugging (25%): Debugging involves stripping all subqueries and consolidating the logic. The `Student` and `Department` tables are joined once via `INNER JOIN`. The filtering condition from the `IN` subquery is moved directly into the main query's `WHERE` clause against the joined `Department` table.

Optimization (20%): By refactoring into a single JOIN, we reduce the time complexity from potentially $O(N * M^3)$ in the worst unoptimized execution plan to $O(N + M)$ using Hash Joins. Furthermore, creating indexes on `Student.DeptID` and `Department.DeptName` ensures the filtering and joining operations occur in memory nearly instantly.

Analysis & Reasoning (20%): Relational databases are optimized to resolve relations in the FROM clause, not the SELECT clause. Flattening the query structure mathematically reduces the number of relational algebraic operations the engine has to process, leading to a much cleaner and logically sound execution plan.

Code Quality & Solution (15%):

-- Highly Optimized Consolidated Query

-- Suggested Index: CREATE INDEX idx_dept_name ON Department(DeptName);

```
SELECT
    s.Name,
    s.Marks,
    d.DeptName,
    d.HOD
FROM
    Student s
INNER JOIN
    Department d ON s.DeptID = d.DeptID
```

WHERE

d.DeptName IN ('Computer Science', 'Electronics');

Q10: Debug relational calculus expression quantifier

Problem Identification (20%): The Relational Calculus expression uses the Universal Quantifier ($\forall t$). This strict logic dictates that a student 's' will only be returned if their marks are strictly greater than EVERY single student 't' in Department 2. If even one student in Dept 2 has higher marks, it evaluates to false. The prompt specifically requested 'AT LEAST ONE'.

Debugging (25%): To fix this, the Universal Quantifier ($\forall$) must be changed to the Existential Quantifier ($\exists$). Furthermore, when using the existential quantifier, the logical implication operator ($\rightarrow$) must be replaced with the logical AND operator ($\wedge$). Implication with $\exists$ leads to vacuously true statements if no students exist in Dept 2.

Optimization (20%): Evaluating existential quantifiers is computationally cheaper for a database engine than universal quantifiers. For $\exists$, the engine can short-circuit and stop searching the moment it finds the first student in Dept 2 with lower marks ($O(1)$ best case). For $\forall$, it must scan the entire set to ensure no violations exist ($O(N)$ best case).

Analysis & Reasoning (20%): In formal logic and tuple relational calculus, 'for all' ($\forall$) coupled with implication ($\rightarrow$) defines absolute boundaries. 'There exists' ($\exists$) coupled with conjunction ($\wedge$) defines partial intersections. Matching the mathematical quantifier strictly to the plain-English business requirement is critical for accurate data retrieval.

Code Quality & Solution (15%):

-- Corrected Tuple Relational Calculus Expression

{ s.Name | Student(s) $\wedge$ ($\exists t$) (Student(t) $\wedge$ t.DeptID = 2 $\wedge$ s.Marks > t.Marks) }